

C200 PROGRAMMING ASSIGNMENT № 9

Dr. M.M. Dalkilic

Computer Science

School of Informatics, Computing, and Engineering

Indiana University, Bloomington, IN, USA

November 17, 2024

Introduction

In this homework, you'll work on translating critical thinking to programming. You will have to read about sqlite3: <https://docs.python.org/3/library/sqlite3.html>.

Commit, push and submit to Autograder by **11:00 PM EDT, Saturday, November 23, 2024**

1. Make sure that you are **following the instructions** in the PDF, especially the format of output returned by the functions. For example, if a function is expected to return a numerical value, then make sure that a numerical value is returned (not a list or a dictionary). Similarly, if a list is expected to be returned then return a list (not a tuple, set or dictionary).
2. Test **debug** the code well (syntax, logical and implementation errors), before submitting to the Autograder. These errors can be easily fixed by running the code in VSC and watching for unexpected behavior such as, program failing with syntax error or not returning correct output.
3. Make sure that the **code does not have infinite loop (that never exits) or an endless recursion (that never completes)** before submitting to the Autograder. You can easily check for this by running in VSC and watching for program output, if it terminates timely or not.
4. Given that you already tried points 1-3, if you see that Autograder does not do anything (after you press 'submit') and waited for a while (30 seconds to 50 seconds), try refreshing the page or using a different browser.
5. Once you are done testing your code, comment out the tests i.e. the code under the `__name__ == "__main__"` section.

Problem 1: Random Walk

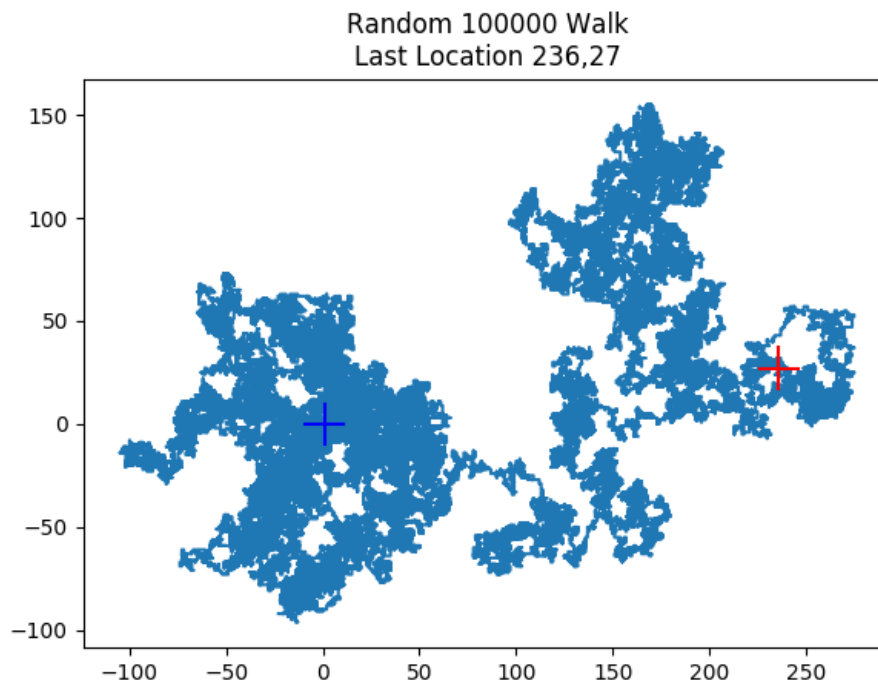


Figure 1: The blue cross is where we started and the red cross is where we ended.

A random walk is a *stochastic process*. A stochastic process is a series of values that are not determined functionally, but probabilistically. The random walk is supposed to describe an inebriated person who, starting from the bar, intends to walk home, but because of intoxication instead randomly takes single steps either forward or backward, left or right. The person has no memory of any steps taken, so theoretically, the person shouldn't move too far from where he or she starts. Random walks are used to model many phenomena, like the size of the web or changes in financial instruments. We will model a 2D random walk with two arrays x and y where x represents moving left or right and y represents forward or backward. The index i will represent the step and $x[i], y[i]$ will represent the location at step i . So, for $i=0$, we have $x[0], y[0]$ (starting place). Using random we choose from the list $[1,2,3,4]$. If the value is one then we move right from the previous x position:

```
1 x[i] = x[i-1] + 1
2 y[i] = y[i-1]
```

If the value is two, then we move left from the previous x position:

```
1 x[i] = x[i-1] - 1
2 y[i] = y[i-1]
```

If the value is three, we move up from the previous y position:

```

1 x[i] = x[i-1]
2 y[i] = y[i-1] + 1

```

And when the value is four, we move down from the previous y position:

```

1 x[i] = x[i-1]
2 y[i] = y[i-1] - 1

```

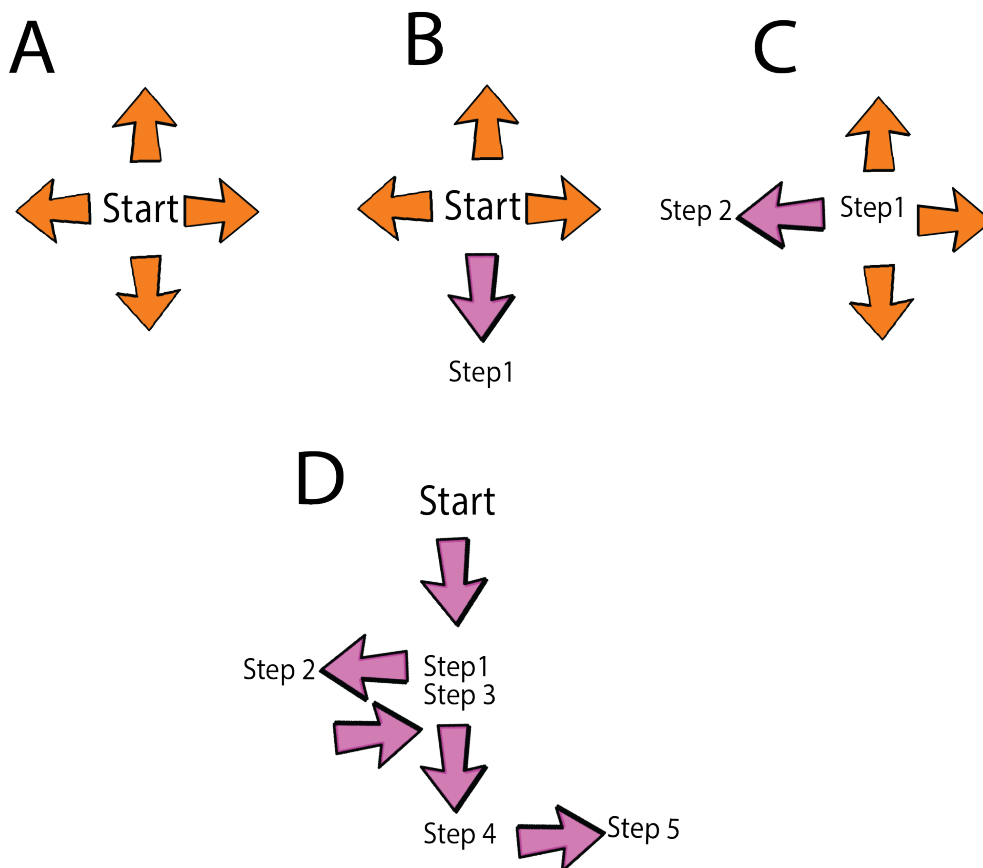


Figure 2: The first few steps of a random walk.

Here is another way to describe this:

$$step_i(n) = \begin{cases} \text{left} & step(n-1), i = 1 \\ \text{right} & step(n-1), i = 2 \\ \text{up} & step(n-1), i = 3 \\ \text{down} & step(n-1), i = 4 \end{cases} \quad (1)$$

Deliverable for Problem 1

- Complete the `step` function. (Template code has been provided)
- These are random walks, these outputs will be different
- Comment the import of `matplotlib` before submitting to the Autograder

Problem 2: Line Class

In this problem, we will build a 2D line class. The starter file already contain the following skeleton code:

```
1 class Line:
2     def __init__(self, **kwargs):
3         # should have self.b, self.m, self.fn
4         pass
5
6     def build_lambda(self):
7         pass
8
9     def sign(self):
10        if self.b > 0:
11            return f"+ {round(self.b,2)}"
12        elif self.b < 0:
13            return f"- {round(abs(self.b),2)}"
14        else:
15            return ""
16
17    def get_slope_intercept(self):
18        return self.m, self.b
19
20    def __and__(self, right_line):
21        pass
22
23    def __call__(self, arg):
24        pass
25
26    def __str__(self):
27        return f"y = {'' if self.m == 1 else round(self.m,2)}x {self.↵
            sign()}"
```

We'll now discuss each method that you have to complete.

(1) The `__init__` allows for four different kinds of initializations using this form of a line ($y = mx + b$)

- two points
- point and intercept
- point slope
- function where input is a string showing equation without `**`

At least three attributes `self.m`, `self.b`, `self.fn` that hold the slope, intercept, and λ functional form of the line. Here are examples of each:

```
1 x = Line(points = ((-2,3),(3,5)))
2 y = Line(point_intercept = ((-2,3), 3.8))
3 z = Line(function = '0.4x + 3.8')
4 a = Line(point_slope = ((-2,3),.4))
5 b = Line(function = 'x - 3')
```

(2) The `build_lambda` uses `self.m`, `self.b` and returns `self.fn` ($\lambda x : xm + b$) so that we can use the line as a function. You are free to use this in `__init__` if you deem it useful.

(3) The `__and__` allows us to override `&`. This is used to determine if we have two lines: if they intersect, then a tuple returned with the coordinates of the point of intersection; if lines are parallel then the tuple ("parallel",) is returned; if lines are the same then tuple ("same",) is returned.

(4) The `__call__` function allows us to use the objects as a function call. For example,

```
1 x = Line(points = ((-2,3),(3,5)))
2 print(x(2),x)
```

produces:

```
1 4.6 y = 0.4x + 3.8
```

since $0.4(2) + 3.8 = .8 + 3.8 = 4.6$. You are encouraged to visit python.org to read about this overload.

The following code:

```
1 x = Line(points = ((-2,3),(3,5)))
2 y = Line(point_intercept = ((-2,3), 3.8))
3 z = Line(function = '0.4x + 3.8')
4 a = Line(point_slope = ((-2,3),.4))
5 b = Line(function = 'x - 3')
6
7 print(x(2),x)
8 print(y(2),y)
9 print(z(2),z)
10 print(a(2),a)
11 print(b(2),b)
12
13 M1 = Line(function = 'x + 3')
14 N2 = Line(function = '-2x + 12')
15 P3 = Line(function = '-2x + 11')
16 print(M1 & N2)
17 print(M1 & M1)
18 print(N2 & P3)
19
20 l1 = Line(function = 'x + 1')
21 l2 = Line(function = 'x - 5')
22 print(f"{l1} intersect {l2} yields {l1 & l2}")
```

produces:

```
1 4.6 y = 0.4x + 3.8
2 4.6 y = 0.4x + 3.8
3 4.6 y = 0.4x + 3.8
4 4.6 y = 0.4x + 3.8
5 -1 y = x - 3
6 (3.0, 6.0)
7 ('same',)
8 ('parallel',)
9 y = x + 1 intersect y = x - 5 yields ('parallel',)
```

Deliverable for Problem 2

- Complete the Line class
- Do *NOT* use any built-in packages.

Problem 3: SQL

In class we were introduced to SQL and the relational model. You will have a great deal of freedom with this problem. Create a table called `Weather` with attributes `City`, `State`, `High`, `Low` and populate it with the data shown in Table 1. I've made equivalent list comprehension on this

Weather			
City	State	High	Low
Phoenix	Arizona	105	90
Tucson	Arizona	101	92
Flag Staff	Arizona	105	90
San Diego	California	77	60
Albuquerque	New Mexico	80	72
Nome	Alaska	64	-54

Table 1: Relation Weather and tuples

iterable:

```
1 data = [('Phoenix', 'Arizona', 105, 90),
2         ('Tucson', 'Arizona', 101, 92),
3         ('Flag Staff', 'Arizona', 105, 90),
4         ('San Diego', 'California', 77, 60),
5         ('Albuquerque', 'New Mexico', 80, 72),
6         ('Nome', 'Alaska', 64, -54)]
```

You should read about these SQL functions (**NOTE**: SQL functions, not Python functions): `count()`, `sum()`, `min()`, `max()` as well as “group by” and “in”. We’ve given the answer to query 1 (i.e. MySQL query). The results of these queries are shown in the output. The following section shows the results obtained by using python and list comprehension—you are required to implement the MySQL operations in python to obtain the same results (as obtained by the list comprehension).

1. Select all the tuples (Query 1) This is an example—observe where the SQL is and replicate this for the remainder of the queries

```
1 print("Query 1")
2 for i in my_cursor.execute("SELECT * FROM Weather"):
3     print(i)
4 print("List Comprehension: ", data)
```

2. Select all the tuples where the High temperature is less than 80 (Query 2)

```

1 print("Query 2")
2 #for i in my_cursor.execute("Some fantastic SQL"):
3     # pass
4 print("List Comprehension: ", [d for d in data if d[2] < 80 ])

```

3. Select All the cities where the low temperature is strictly greater than the Low of Albuquerque – you cannot use the number 72.0 in the query (Query 3)

```

1 print("Query 3")
2 #for i in my_cursor.execute("Some fantastic SQL"):
3     # pass
4
5 x =[d[0] for d in data if d[3] > [d[3] for d in data if d[0] == 'Albuquerque'][0]]
6 print("List Comprehension: ",x)

```

4. Select the city and temperature with the smallest low temperature (Query 4)

```

1 print("Query 4")
2 #for i in my_cursor.execute("Some fantastic SQL"):
3     # pass
4
5 print("List Comprehension: ",[(d[0],d[3]) for d in data if d[3] in←
    (sorted(data, key = lambda x:x[3])[0])])

```

5. Select the city temperature with the largest high temperature—since there are two, both cities should be returned. (Query 5)

```

1 print("Query 5")
2 #for i in my_cursor.execute("Some fantastic SQL"):
3     # pass
4
5 print("List Comprehension: ",[(d[0],d[2]) for d in data if d[2] in←
    (sorted(data, key = lambda x:x[2],reverse=True)[0])])

```

6. Display the *average* High and Low temperatures—you are not allowed to use Avg() (Query 6)

```

1 print("Query 6")
2 #for i in my_cursor.execute("Some fantastic SQL"):
3     # pass

```

```

4
5 print("List Comprehension: ", [(sum([d[2] for d in data])/len(data)↵
    ),sum([d[3] for d in data])/len(data)])])

```

7. Give the counts of cities by their Low temperatures (Query 7)

```

1 print("Query 7")
2 #for i in my_cursor.execute("Some fantastic SQL"):
3     # pass
4
5 print([(i,list(map((lambda x: x[3]),data)).count(i)) for i in set(↵
    map((lambda x: x[3]),data))])

```

Output

```

Query 1
('Phoenix', 'Arizona', 105.0, 90.0)
('Tucson', 'Arizona', 101.0, 92.0)
('Flag Staff', 'Arizona', 105.0, 90.0)
('San Diego', 'California', 77.0, 60.0)
('Albuquerque', 'New Mexico', 80.0, 72.0)
('Nome', 'Alaska', 64.0, -54.0)
List Comprehension:
[('Phoenix', 'Arizona', 105, 90),
 ('Tucson', 'Arizona', 101, 92),
 ('Flag Staff', 'Arizona', 105, 90),
 ('San Diego', 'California', 77, 60),
 ('Albuquerque', 'New Mexico', 80, 72),
 ('Nome', 'Alaska', 64, -54)]

```

```

Query 2
('San Diego', 'California', 77.0, 60.0)
('Nome', 'Alaska', 64.0, -54.0)
List Comprehension:
[('San Diego', 'California', 77, 60),
 ('Nome', 'Alaska', 64, -54)]

```

```

Query 3
('Phoenix',)
('Tucson',)
('Flag Staff',)
List Comprehension:  ['Phoenix', 'Tucson', 'Flag Staff']

```

Output

Query 4

('Nome', -54.0)

List Comprehension: [('Nome', -54)]

Query 5

('Phoenix', 105.0)

('Flag Staff', 105.0)

List Comprehension: [('Phoenix', 105), ('Flag Staff', 105)]

Query 6

(88.66666666666667, 58.333333333333336)

List Comprehension: [(88.66666666666667, 58.333333333333336)]

Query 7

(-54.0, 1)

(60.0, 1)

(72.0, 1)

(90.0, 2)

(92.0, 1)

List Comprehension: [(72, 1), (-54, 1), (60, 1), (90, 2), (92, 1)]

Deliverable for Problem 3

- Write the MYSQL code for queries 1-7 in their respective functions (inspect the starter code for query1).
- Although this is generally not done, for the purposes of making the problem easier, we will make sure to remove the table and repopulate it everytime the code is run so we don't have to comment out portions.

4: Correlation

For this problem, we fix the number of columns to four. The function `correlation4` takes a list with a header and returns the correlation of all pairwise different columns sorted in ascending order using Pearson's discussed in the class.

The following code:

```
1 print(data4)
2 print(correlation4(data4))
```

produces:

```
1 [('A', 'B', 'C', 'D'),
2  (30, 31, 28, 932), (1, 2, 9, 7), (33, 35, 6, 1127),
3  (14, 15, 18, 213), (5, 7, 3, 33), (7, 9, 29, 61),
4  (19, 21, 15, 383), (10, 12, 16, 113)]
5 [('C', 'D', 0.058999692221547315), ('B', 'C', 0.14030184909443133),
6  ('A', 'C', 0.15133226179357964), ('B', 'D', 0.9701917605618),
7  ('A', 'D', 0.972914603335017), ('A', 'B', 0.999014823479421)]
```

Observe that columns C,D have the lowest correlation.

Correlation

- Implement the function.
- You must use `from scipy.stats import pearsonr` to find the correlation.

5: Clustering

In this problem you'll use `from sklearn.cluster import KMeans`. The function call to build the object is:

```
1 kmeans = KMeans(n_clusters=k, random_state=0, n_init="auto").fit(X)
```

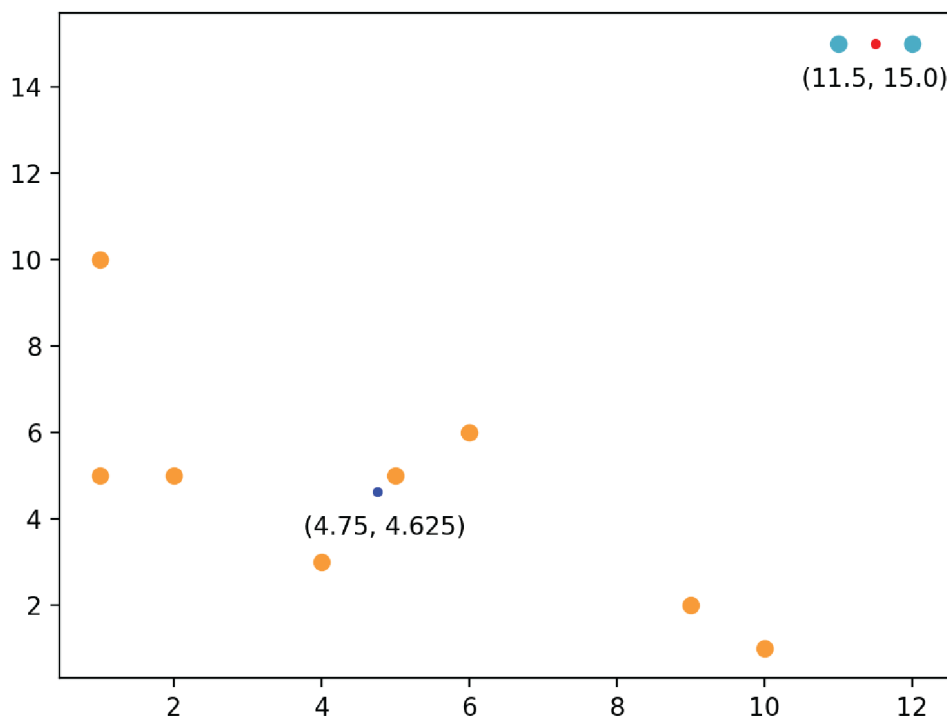
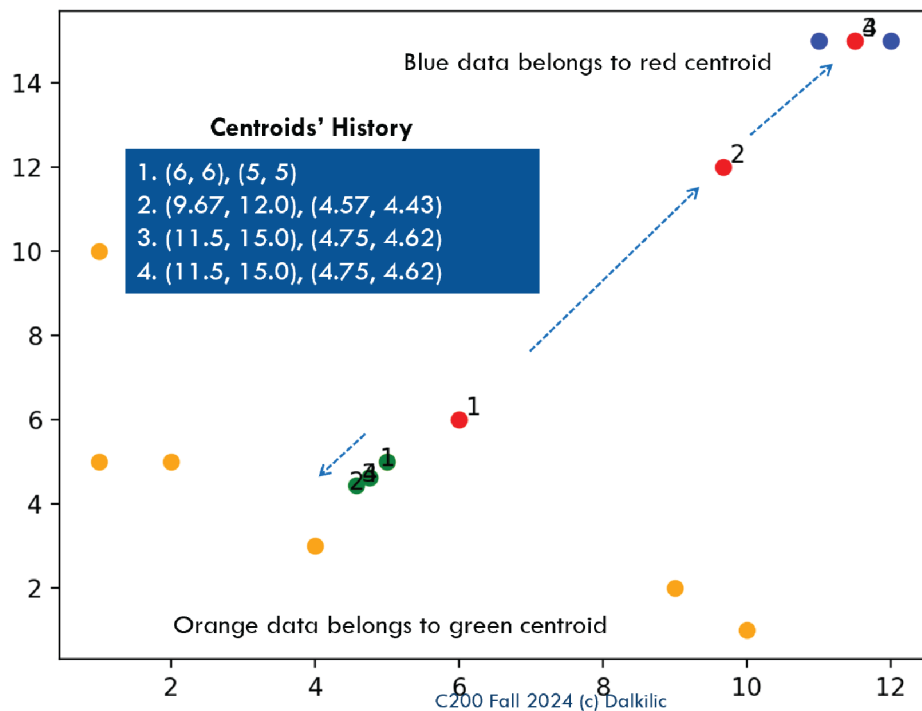
where `X` is the data as an array.

We will start by using the data from the last lecture, and `skcikit-learn`, Fig.3, shows the plot of clusters. Next, you'll use the data in the previous problem but selecting the two, least correlated columns. The function `my_cluster(data, k)` takes data as a list without a header and the number of blocks `k`. It returns the clustering object build. I've added some code to plot a 2D clustering. You can uncomment this when youve finished the function.

Deliverable for Problem 4

- Implement the function.
- You will handcode the two least correlated columns from the previous problem and return a cluster object.

My k-means



scikit-learn kmeans

Figure 3: Comparing k -means: mine and `scikit-learn`. Observe that the values are essentially identical.

Programming Partners

bronburk@iu.edu, sxanders@iu.edu
yaljabri@iu.edu, muksingh@iu.edu
keswar@iu.edu, omwells@iu.edu
jdkakare@iu.edu, zz67@iu.edu
mahichan@iu.edu, ishyadav@iu.edu
btbasing@iu.edu, jjrinald@iu.edu
hdjour@iu.edu, bsati@iu.edu
nifredri@iu.edu, kalerobi@iu.edu
benchart@iu.edu, bobbyun@iu.edu
kadlakha@iu.edu, aamuse@iu.edu
ramgowda@iu.edu, adimaran@iu.edu
ag38@iu.edu, jvmenach@iu.edu
mgbekele@iu.edu, ibshaw@iu.edu
evbutts@iu.edu, youngct@iu.edu
alurs@iu.edu, rheashah@iu.edu
jbriar@iu.edu, nguyen17@iu.edu
kevgross@iu.edu, mzuhair@iu.edu
jagoodal@iu.edu, rvasude@iu.edu
bahls@iu.edu, tlichten@iu.edu
mbenites@iu.edu, jasreese@iu.edu
ajkouts@iu.edu, stewrach@iu.edu
jigarnes@iu.edu, mmmatias@iu.edu
ojallow@iu.edu, tersacks@iu.edu
heoji@iu.edu, ksabloak@iu.edu
asgurram@iu.edu, missavin@iu.edu
btcunnin@iu.edu, sunathan@iu.edu
tborder@iu.edu, neenpate@iu.edu
abondada@iu.edu, jiajtang@iu.edu
okirchen@iu.edu, nmeely@iu.edu
elscheng@iu.edu, vpremku@iu.edu
adagar@iu.edu, wschrama@iu.edu
felkhatt@iu.edu, jakpatel@iu.edu
clickc@iu.edu, roldenbu@iu.edu
ajafowl@iu.edu, isvoor@iu.edu
damaatki@iu.edu, sholove@iu.edu
agrief@iu.edu, anangla@iu.edu
jb108@iu.edu, cardnorr@iu.edu
jkittur@iu.edu, aramanu@iu.edu
jejea@iu.edu, ozsachs@iu.edu

akoduru@iu.edu, jakepars@iu.edu
dh43@iu.edu, nealsing@iu.edu
ljhorman@iu.edu, angzamora@iu.edu
ghargro@iu.edu, sahasrir@iu.edu
adekolaw@iu.edu, rvenigal@iu.edu
meiyhe@iu.edu, irodeman@iu.edu
idasilva@iu.edu, ozopich@iu.edu
ethdeatr@iu.edu, westje@iu.edu
natkwon@iu.edu, marcmarq@iu.edu
ethhanna@iu.edu, mjo4@iu.edu
adj18@iu.edu, manpate@iu.edu
jaccryst@iu.edu, adivaidy@iu.edu
ckbarts@iu.edu, aidpride@iu.edu
mabana@iu.edu, antonegr@iu.edu
ahmealsh@iu.edu, prmundra@iu.edu
nwconn@iu.edu, vivitran@iu.edu
cgcheane@iu.edu, kenmoses@iu.edu
jahjacks@iu.edu, martibr@iu.edu
joglickm@iu.edu, camoverm@iu.edu
xbchrist@iu.edu, samyadav@iu.edu
edeitchl@iu.edu, ctsvetan@iu.edu
ttfender@iu.edu, evanmayo@iu.edu
bedani@iu.edu, cjrodenb@iu.edu
aabdulw@iu.edu, mt87@iu.edu
sdondeti@iu.edu, tunnitha@iu.edu
mkulekci@iu.edu, cpmann@iu.edu
drmiguth@iu.edu, dematt@iu.edu
pchitwoo@iu.edu, fasrasoo@iu.edu
addfishe@iu.edu, corawang@iu.edu
florech@iu.edu, olsoncs@iu.edu
reclay@iu.edu, sstickl@iu.edu
lddial@iu.edu, mmaplet@iu.edu
clarkina@iu.edu, djtheodo@iu.edu
vkabra@iu.edu, zhangsuo@iu.edu
rdotsu@iu.edu, wl52@iu.edu
zkabacin@iu.edu, elwesley@iu.edu
smhamrah@iu.edu, tanilitt@iu.edu
sabyrap@iu.edu, maymat@iu.edu
aarepal@iu.edu, emasseng@iu.edu
nantinao@iu.edu, nmenne@iu.edu
coubenne@iu.edu, pp22@iu.edu

ldeel@iu.edu, esspatel@iu.edu
garmenda@iu.edu, conaitis@iu.edu
mugabr@iu.edu, matyun@iu.edu
radutta@iu.edu, seuryu@iu.edu
cjo3@iu.edu, cjlomax@iu.edu
rycostin@iu.edu, risvphil@iu.edu
am280@iu.edu, soyedele@iu.edu
pranchan@iu.edu, smitfi@iu.edu
keswa@iu.edu, dommonte@iu.edu
drussow@iu.edu, hmahapat@iu.edu
rbernfel@iu.edu, anryoun@iu.edu
armstan@iu.edu, mszach@iu.edu
michdool@iu.edu, alzemlya@iu.edu
zahaider@iu.edu, vnathany@iu.edu
tbashee@iu.edu, atsarver@iu.edu
dajuhnke@iu.edu, jl367@iu.edu
mdevara@iu.edu, pjminne@iu.edu
lafly@iu.edu, fsafianu@iu.edu
jkuk@iu.edu, amarynow@iu.edu
skapure@iu.edu, prnemani@iu.edu
blstcoop@iu.edu, mahonm@iu.edu
nikdevra@iu.edu, ssp4@iu.edu
jjithesh@iu.edu, jmsequei@iu.edu
jk314@iu.edu, marsh1@iu.edu
jaeblank@iu.edu, kalovert@iu.edu
aare@iu.edu, tnerkar@iu.edu
ahsans@iu.edu, aposton@iu.edu
jamcham@iu.edu, mawlorch@iu.edu
pikhatri@iu.edu, smitaa@iu.edu
efoltyn@iu.edu, hshyama@iu.edu
cammlanc@iu.edu, kiloscot@iu.edu
kbussel@iu.edu, whitedyl@iu.edu
etferba@iu.edu, lukthiel@iu.edu
tycarmo@iu.edu, ijrussel@iu.edu
nfarzane@iu.edu, rvarrier@iu.edu
jbosio@iu.edu, adenyu@iu.edu
vicbarre@iu.edu, msharsh@iu.edu
byersjac@iu.edu, dajshaw@iu.edu
concraft@iu.edu, ydpatil@iu.edu
maboncos@iu.edu, aishravi@iu.edu
jlaytin@iu.edu, thebmill@iu.edu

alneyadk@iu.edu, mjseymou@iu.edu
nbriere@iu.edu, mitcal@iu.edu
herncr@iu.edu, drimawi@iu.edu
iblay@iu.edu, braxluns@iu.edu
dakomi@iu.edu, lisalaza@iu.edu
heissler@iu.edu, tkthang@iu.edu
rogoenka@iu.edu, everma@iu.edu
banksda@iu.edu, ndmyer@iu.edu
rgyasini@iu.edu, ncw1@iu.edu
dbarbari@iu.edu, isapittm@iu.edu
agotanco@iu.edu, auskirvi@iu.edu
yc127@iu.edu, liaobrie@iu.edu
zalsamri@iu.edu, sandro@iu.edu
xmdrew@iu.edu, gl25@iu.edu
dayinla@iu.edu, lemarc@iu.edu
shkami@iu.edu, aralover@iu.edu
adatwani@iu.edu, davepais@iu.edu
bgelard@iu.edu, taylbryo@iu.edu
mjfarrin@iu.edu, mkreddy@iu.edu
laigrant@iu.edu, kzerbino@iu.edu
ealfalah@iu.edu, aluckhau@iu.edu
nafike@iu.edu, jacloise@iu.edu
jackchap@iu.edu, bawsung@iu.edu
bthenson@iu.edu, kevwong@iu.edu
adguhan@iu.edu, jrwaren@iu.edu
jhl14@iu.edu, shwanara@iu.edu
ndodoh@iu.edu, navann@iu.edu
chfurlow@iu.edu, hsh1@iu.edu
chjflor@iu.edu, kr15@iu.edu
carthugh@iu.edu, jopantoj@iu.edu
akeshav@iu.edu, negia@iu.edu
agahlau@iu.edu, olesmith@iu.edu
bcarpine@iu.edu, ansjmeht@iu.edu
adodaro@iu.edu, cmolaski@iu.edu
mgoenka@iu.edu, gsackie@iu.edu
abrking@iu.edu, jawegman@iu.edu
sydforem@iu.edu, roberjuv@iu.edu
grhamlin@iu.edu, bgtruong@iu.edu
sergonza@iu.edu, sumuth@iu.edu
jergalan@iu.edu, pmiesch@iu.edu
ygajjar@iu.edu, cpmathew@iu.edu

crigonza@iu.edu, zs15@iu.edu
enkarl@iu.edu, kokulski@iu.edu
srkagama@iu.edu, aidsteph@iu.edu
mthinman@iu.edu, dmonroyh@iu.edu
ebederma@iu.edu, lydiwang@iu.edu
nicdelg@iu.edu, cs155@iu.edu
bdgonzal@iu.edu, klohiya@iu.edu
skalluru@iu.edu, hailreid@iu.edu
iyersan@iu.edu, rosanghv@iu.edu
egilmour@iu.edu, kpazdur@iu.edu
mahchaud@iu.edu, jskains@iu.edu
langarbe@iu.edu, cmellgre@iu.edu
mailic@iu.edu, robreese@iu.edu
mfbah@iu.edu, clushell@iu.edu
cdasari@iu.edu, anupand@iu.edu
eabilius@iu.edu, ssanjee@iu.edu
skyangel@iu.edu, dyllopez@iu.edu
talfares@iu.edu, mmalviya@iu.edu
ldeveau@iu.edu, jwubelho@iu.edu
nimjain@iu.edu, vpottete@iu.edu
blbinder@iu.edu, slopezve@iu.edu
cedrchen@iu.edu, nshnoble@iu.edu
boydmad@iu.edu, cmekat@iu.edu
dgiffor@iu.edu, froy@iu.edu
adhimat@iu.edu, anrajput@iu.edu
owencape@iu.edu, ainlukas@iu.edu
eberens@iu.edu, rnalluri@iu.edu
rkoston@iu.edu, jnimmala@iu.edu
evarcarm@iu.edu, bogawa@iu.edu
mrb9@iu.edu, reshetty@iu.edu
ehchoo@iu.edu, isundara@iu.edu
salvchri@iu.edu, joevu@iu.edu
nitiarun@iu.edu, rasahu@iu.edu
natkling@iu.edu, audscha@iu.edu
ssdatla@iu.edu, jw363@iu.edu
timolaws@iu.edu, rpolu@iu.edu
gkreutes@iu.edu, sasubb@iu.edu
twchase@iu.edu, athason@iu.edu